Catamaran Data Hub: Build Spec

Oct 18, 2026 · @Michael Kipp

Instructions for the coding agent

Build a self-hosted web app that runs as a Home Assistant add-on on a Home Assistant OS (HAOS) box aboard a sailing catamaran. A Signal K add-on on the same box reads the boat's NMEA networks. This app reads from Signal K and provides custom dashboards, an automatic hourly logbook, maintenance tracking, anchor watch and checklists. People reach it through HA ingress: in the HA companion app on phones, or in a browser at the nav station. It must work with no internet connection and keep CPU load low. The hardware lives in a hot, closed marine space, and thermal throttling counts as a failure. This document is the complete build spec. Where it's silent, pick the simplest option that meets the stated behavior and note the choice in `DECISIONS.md`.

Ground rules

1. **Never write to the NMEA bus.** The app writes to Signal K only under `notifications.app.*` and the derived paths listed in this spec. No autopilot commands, no PUT requests to device paths.
2. **Offline first.** Once the add-on image is installed, nothing needs the internet. Fonts, icons and libraries are bundled. No CDNs.
3. **Light on the CPU.** Sample each path at the lowest rate that meets the need, batch database writes, and never build or compile on the device. Never mirror Signal K data into HA's entities or recorder beyond the summary entities in this spec.
4. **SI in, user units out.** Signal K values are SI. Store SI. Convert only at display time.
5. **Missing beats wrong.** A stale or absent value displays as stale or missing and is stored as `null`. Never substitute the last known value silently.
6. **GPS time for the log.** Log timestamps come from Signal K's `navigation.datetime`, not the host clock. Offshore, HAOS has no time source.
7. **The Pi raises safety alarms, not the browser.** Alarm evaluation runs in the backend. A locked phone is not an alarm system.
8. **Log entries are append-only.** Corrections are amendment rows. No `UPDATE` on a signed entry's snapshot, ever.

9. **Match the mockups.** The design canvas is the visual source of truth. Tokens and rules are in the Design system section. Don't introduce new colors, fonts or component styles without a `DECISIONS.md` entry.
10. TypeScript strict mode everywhere. No `any` without a comment explaining why.
11. Every milestone ends with its acceptance tests passing. Don't start the next milestone until they do.

Build order

Work through the milestones at the end of this document in order: Foundation, Dashboards, Logbook, Maintenance, Anchor watch and checklists, Hardening. Each lists its acceptance criteria.

Reference material

- Design mockups: the "Catamaran Data Hub UI" canvas, eight screens. Export the `.dc.html` files into `design/mockups/` in the repo before starting UI work.
- Product spec: "Catamaran Data Hub: Product Spec" (the why behind every feature). This build spec overrides it where they differ.

Stack and packaging

Data Hub ships as one Home Assistant add-on: a single container holding a Fastify backend that also serves the built React app. It runs beside the Signal K add-on on the same HAOS host. HA provides ingress (TLS and authentication), backups, updates and the watchdog, so the app doesn't build any of those.

Layer	Choice
Host	HAOS on a Raspberry Pi 5 (8 GB, NVMe boot) or a fanless x86 mini PC. Support both: build `aarch64` and `amd64` images
Data source	Signal K server add-on on the same host
Packaging	HA add-on: `config.yaml`, Dockerfile on a slim Node base image. CI builds multi-arch images and publishes them to GHCR. Nothing is compiled on the device

Layer	Choice
Runtime	Node.js 22 LTS, TypeScript 5, ESM
Backend	Fastify, `@fastify/websocket`, `better-sqlite3` (native module built in CI), `ws` for the Signal K client
Validation	Zod schemas in `packages/shared`, used by both API and UI
Database	SQLite under the add-on's `/data`, WAL mode, batched writes, numbered `.sql` migrations run at startup
Frontend	React 18, Vite with a relative `base` (required for ingress), React Router, TanStack Query, a small Zustand store for live values
Dashboard grid	`react-grid-layout`
Charts	`uPlot`
Fonts	`@fontsource/barlow`, `@fontsource/barlow-semi-condensed`, `@fontsource/ibm-plex-mono`, bundled
PDF export	`pdfkit`, generated on demand only
HA integration	Supervisor API from inside the add-on (`SUPERVISOR_TOKEN`): host shutdown, a few summary entities, companion app notifications
Tests	Vitest (unit, API), Playwright (UI, mobile and desktop viewports, under an ingress-style path prefix)

Add-on config

Starting point for `addon/config.yaml`. Verify every key against the current HA add-on docs before relying on it.

```
name: Data Hub
version: "0.1.0"
slug: data_hub
description: Dashboards, logbook and maintenance for the boat
```

```
arch: [aarch64, amd64]
image: ghcr.io/OWNER/data-hub-{arch}
init: false
ingress: true
ingress_port: 8099
panel_icon: mdi:sail-boat
panel_title: Data Hub
hassio_api: true
hassio_role: manager          # needed for host shutdown on UPS low voltage
homeassistant_api: true       # summary entities and notifications
gpio: true                    # sounder and UPS signal pins
watchdog: "http://[HOST]:[PORT:8099]/api/v1/health"
backup: hot
backup_pre: /usr/bin/data-hub-snapshot
backup_exclude:
  - "data-hub.db"
  - "data-hub.db-wal"
  - "data-hub.db-shm"
options:
  signalk_url: "http://SIGNALK_ADDON_HOST:3000"
  sounder_gpio: null
  ups_gpio: null
  log_level: info
schema:
  signalk_url: url
  sounder_gpio: "int?"
  ups_gpio: "int?"
  log_level: list(debug|info|warn|error)
```

The backup approach: HA takes a hot backup while logging continues. `backup_pre` writes a consistent copy with SQLite's online backup API to `/data/snapshot/data-hub.db`, and the live WAL files are excluded. On startup, if the live database is missing and a snapshot exists, the backend restores from the snapshot. This is how an HA restore brings the database back.

Repo layout

```
data-hub/
  repository.yaml        HA add-on repository manifest
  addon/
    config.yaml
    Dockerfile
```

```
    run.sh               reads /data/options.json, starts the backend
    icon.png, logo.png
    translations/en.yaml
  apps/
    server/              Fastify backend
      src/
        signalk/         client, subscription manager, rate classes,
staleness tracker
        recorder/        buffered history writes and downsampling
        logbook/         scheduler, triggers, snapshot builder, GPS time,
PDF/CSV/GPX export
        maintenance/     due calculation, inventory
        alarms/          threshold evaluation, anchor watch, notification
writer, sounder output
        derived/         derived value calculators
        ha/              Supervisor client: shutdown, entities, notifications
(with a fake for tests)
        api/             REST routes, WebSocket hub, ingress auth
        db/              migrations/, repositories, snapshot
      test/
    web/                 React app
      src/
        shell/  dashboards/  logbook/  maintenance/  anchor/  checklists/
settings/  theme/
      test/
  packages/
    shared/              zod schemas, types, units.ts, paths.ts
  design/
    mockups/             exported .dc.html files from the canvas
  .github/workflows/     build, test, multi-arch image publish
  DECISIONS.md
```

Dev environment

- Run Signal K locally in Docker with its bundled sample data (the server has `--sample-nmea0183-data` and `--sample-n2k-data` startup flags). Confirm the flags against the current Signal K docs.
- The backend talks to HA only through an `ha/` interface with a fake implementation, so it runs outside HA in dev and tests. Ingress headers are simulated by a small dev proxy that mounts the app under a random path prefix and injects user headers.

- A scripted replay mode for tests: `apps/server/test/fixtures/*.jsonl` holds recorded Signal K deltas with timestamps, replayed by a fake Signal K WebSocket server at 1x or accelerated speed. The logbook, alarm, anchor and GPS-time tests depend on it.
- For end-to-end checks, install the add-on from the repository on a HAOS VM.
- `pnpm dev` starts Signal K (Docker), the dev proxy, the backend with watch, and Vite.

Design system

Everything here is lifted from the mockups. Implement it as CSS custom properties set on `:root` by theme (`data-theme="day"` or `"night"`), plus a `tokens.ts` that mirrors them for SVG widgets. Components never use raw hex values.

Color tokens

Token	Day	Night	Use
`--bg`	#EEF2F3	#000000	App background
`--panel`	#FFFFFF	#0B0302	Tiles, cards, nav bars
`--line`	#D3DBDE	#33100D	Borders, dividers
`--ink`	#0E1A22	#E0392D	Primary text and values
`--sub`	#4F5F69	#A8322A	Labels, secondary text
`--accent`	#0D5A85	#E0392D	Active tab, primary buttons, selection
`--accent-tint`	#E1EBF1	#1A0605	Selected backgrounds, info boxes
`--accent-ink`	#0B4A6E	#E0392D	Text on accent tint
`--good`	#1F7A5A	#A8322A	Connected dots, OK status
`--warn`	#9A5300	#FF5A4E	Warning text and borders, due now, unsigned
`--warn-tint`	#FBF0DF	#1A0605	Warning row backgrounds
`--alarm`	#B42318	#FF4A3D	Alarm and overdue
`--alarm-tint`	#FBE9E7	#1A0605	Overdue backgrounds

Token	Day	Night	Use
`--alarm-banner-bg`	#B42318	#FF4A3D	Full-width alarm banner
`--alarm-banner-ink`	#FFFFFF	#000000	Text on the alarm banner
`--stale`	#6F7B82	#5C1A15	Stale data text and dashed border
`--port`	#C0362C	#A8322A	Port side values
`--stbd`	#1F7A5A	#E0392D	Starboard side values
`--nav-bg`	#0E1A22	#000000	Desktop sidebar
`--nav-active`	#1E3240	#1A0605	Desktop sidebar active item

Color rules:

- Red means alarm. Amber means warning or due. Neither is decorative.
- Status is never color alone: every status also has a text label ("UNSIGNED", "Overdue", "Stale").
- Port and starboard also differ by label (P/S, PORT/STBD) and, in charts, by line style (port solid, starboard dashed).
- Night mode collapses everything to reds on black. Brightness should be low enough to keep night vision. The alarm banner is the one bright element allowed.

Typography

Role	Font	Size / weight	Notes
UI text	Barlow	14 to 16 px / 400 to 600	Body and controls
Tile label	Barlow	12 px / 600, uppercase, 0.06em tracking	Every widget label
Instrument value, phone	Barlow Semi Condensed	44 px / 600, line-height 1	`font-variant-numeric: tabular-nums`
Instrument value, secondary	Barlow Semi Condensed	18 to 34 px / 600	Stats rows, sub-values

Role	Font	Size / weight	Notes
Unit	Barlow	14 to 15 px / 500, `--sub`	Always beside the value, never omitted
Screen title	Barlow	24 px phone, 32 px desktop / 700	
Signal K paths	IBM Plex Mono	12 to 13 px / 400	Path browser, widget settings

All numeric values use tabular figures so digits don't jump as they update.

Spacing, shape and size

- 8 px base unit. Tile grid gap: 8 px on phone, 16 px on desktop. Screen padding: 12 to 16 px on phone, 24 px on desktop.
- Radius: 14 px tiles and cards, 10 to 12 px inputs and buttons, 999 px pills.
- Touch targets: 44 px minimum everywhere, 56 px for primary actions in the cockpit (Save to log, Acknowledge, Anchor up).
- Borders: 1 px `--line` default, 1.5 px for status emphasis (warn, alarm), 2 px for selection, 1.5 px dashed `--stale` for stale tiles.
- Icons: inline stroke SVG, 1.8 px stroke, 20 to 24 px. No icon fonts, no emoji.

Breakpoints and shell

Name	Width	Shell
phone	under 600 px	Bottom tab bar, 72 px: Dashboards, Log, Maintenance, More
tablet	600 to 1023 px	Bottom tabs in portrait, 240 px sidebar in landscape
desktop	1024 px and up	240 px dark sidebar, content area, optional right panel 320 to 380 px

On every screen at every size: a status strip (Signal K connection, GPS fix, UTC time, passage state), the night-mode toggle, and the alarm banner slot. The alarm banner pushes content down; it never overlays it.

Component inventory

Build these once in `apps/web/src/shell` and `theme` and reuse them: `Tile` (label, value, unit, sub line, states normal / warn / alarm / stale / missing), `StatusPill`, `SegmentedControl`, `IconButton` (44 px, requires `aria-label`), `PrimaryButton`, `TabBar`, `Sidebar`, `StatusStrip`, `AlarmBanner`, `Card`, `FilterChips`, `FormField` (label always visible, never placeholder-only).

Signal K integration

The backend holds the only connection to Signal K. It subscribes to every path the app records or alarms on, at the rate each path actually needs, whether or not anyone is looking. Browsers never talk to Signal K directly. They get live values from the Data Hub WebSocket through ingress, which keeps auth in one place and means one Signal K client instead of one per phone.

Connection

- Connect to the URL in the add-on option `signalk_url`: the Signal K add-on's internal hostname on the HA network, port 3000.
- Stream endpoint: `/signalk/v1/stream?subscribe=none`, then send subscribe messages. REST for snapshots: `/signalk/v1/api/vessels/self`.
- Subscribe message shape, with `period` set from the path's rate class:

```
{"context": "vessels.self", "subscribe": [{"path":
"navigation.speedOverGround", "period": 1000, "policy": "instant"}]}
```

- Reconnect with exponential backoff (1 s doubling to 30 s max). While disconnected, the backend marks every path stale and records a `connection_gap` row with start and end times.
- Auth: the backend registers as a Signal K device using the access request flow (`/signalk/v1/access/requests`) on first run. The skipper approves it once in the Signal K admin UI. Store the token in `/data` with `0600` permissions.

What the app writes to Signal K

The device token needs write access, but the backend only ever sends deltas to these paths:

- `notifications.app.<alarmId>`: alarm state with `state` (`normal`, `warn`, `alarm`, `emergency`), `method` (`["visual", "sound"]`) and `message`. This is how the MFD and any other Signal K consumer hear the app's alarms.
- Derived values (below), under the `performance.*` paths where Signal K defines one, otherwise under a vendor prefix chosen in `DECISIONS.md`.

A unit test asserts that every outbound delta path matches this allowlist.

Path catalog

`packages/shared/paths.ts` defines every path the app knows about, its SI unit, its display quantity (speed, angle, depth, temperature and so on) and its default stale timeout. Catamaran-specific paths use the `<id>` placeholders `port` and `starboard`, configurable in settings to match the boat's actual Signal K instance ids.

Path	SI unit	Quantity	Stale after
`navigation.position`	lat/lon object	position	10 s
`navigation.speedOverGround`	m/s	speed	5 s
`navigation.speedThroughWater`	m/s	speed	5 s
`navigation.courseOverGroundTrue`	rad	angle	5 s
`navigation.headingTrue`	rad	angle	5 s
`navigation.headingMagnetic`	rad	angle	5 s
`navigation.anchor.position`	lat/lon object	position	none
`environment.wind.angleApparent`	rad, signed	wind angle	5 s
`environment.wind.speedApparent`	m/s	speed	5 s
`environment.wind.angleTrueWater`	rad, signed	wind angle	5 s
`environment.wind.speedTrue`	m/s	speed	5 s
`environment.wind.directionTrue`	rad	angle	5 s
`environment.outside.pressure`	Pa	pressure	60 s

Path	SI unit	Quantity	Stale after
`environment.outside.temperature`	K	temperature	60 s
`environment.water.temperature`	K	temperature	60 s
`environment.depth.belowKeel`	m	depth	10 s
`propulsion.<id>.revolutions`	Hz	rpm	5 s
`propulsion.<id>.state`	string	state	30 s
`propulsion.<id>.temperature`	K	temperature	30 s
`propulsion.<id>.oilPressure`	Pa	pressure (engine)	30 s
`propulsion.<id>.runTime`	s	hours	300 s
`propulsion.<id>.fuel.rate`	m^3/s	flow	30 s
`electrical.batteries.<id>.voltage`	V	voltage	30 s
`electrical.batteries.<id>.current`	A	current	30 s
`electrical.batteries.<id>.capacity.stateOfCharge`	ratio 0 to 1	percent	60 s
`electrical.solar.<id>.current`	A	current	60 s
`tanks.<type>.<id>.currentLevel`	ratio 0 to 1	percent	300 s
`steering.autopilot.state`	string	state	30 s

The path browser also lists any path Signal K publishes that isn't in the catalog. Those get a generic numeric or text widget and a stale timeout of 30 s.

Unit conversion

`packages/shared/units.ts` exports pure functions keyed by quantity. Display preferences are per user. Defaults are US:

Quantity	Default display	Options
speed	kn	kn, km/h, m/s

Quantity	Default display	Options
depth	ft	ft, m, fathoms
distance	nm	nm, km
temperature	°F	°F, °C
pressure (barometric)	hPa	hPa, inHg, mbar
pressure (engine)	psi	psi, bar, kPa
volume	gal (US)	gal, L
flow	gal/h	gal/h, L/h
angle	degrees, true	true or magnetic for headings and courses
wind angle	degrees 0 to 180 with PORT / STBD	signed degrees
rpm	rpm (Hz × 60)	fixed
hours	h with one decimal (s ÷ 3600)	fixed
position	degrees and decimal minutes	DMS, decimal degrees

Every conversion has a unit test with a known value in both directions.

Staleness

A `StalenessTracker` (backend and browser each have one) records the last update time per path. A value is stale once its age exceeds the path's timeout. Stale values render with the stale tile style and an age label ("no update 2 min"). The backend records stale fields as `null` in log snapshots and history.

Sample rates

Every catalog path has a rate class. This is the main lever on CPU load, so default to the slowest class that still answers the question.

Class	Signal K `period`	Paths
fast	1 s	Wind angle and speed, SOG, STW, heading, COG, position
medium	5 s	Depth, engine rpm, battery current, autopilot state
slow	30 s	Temperatures, pressures, voltages, SoC, fuel rate, solar current
very slow	60 s	Tank levels, engine run time, engine state

- A path's effective stale timeout is the larger of its catalog value and three times its period.
- Only fast-class paths are kept at 1 s resolution. Everything else is written straight to the 1-minute tier.
- Samples are buffered in memory and flushed in one transaction every 10 s. Log entries, alarms and amendments are written immediately.
- Opening a dashboard never raises a path's rate. Live updates to browsers are batched and capped at 4 per second per client.

Database schema

One SQLite file at `/data/data-hub.db`, WAL mode, `foreign_keys = ON`, `synchronous = FULL` for the log tables (set per connection). Timestamps are ISO 8601 UTC strings. IDs are ULIDs. Attachments are files on disk under `/data/files/`, referenced by path.

This is the starting migration. The agent may add indexes and columns, but not change the append-only rules.

```
-- users and settings
CREATE TABLE user (
  id TEXT PRIMARY KEY, name TEXT NOT NULL,
  role TEXT NOT NULL CHECK (role IN ('skipper','crew','guest')),
  pin_hash TEXT NOT NULL, unit_prefs TEXT NOT NULL DEFAULT '{}',
  created_at TEXT NOT NULL
);
CREATE TABLE setting (key TEXT PRIMARY KEY, value TEXT NOT NULL);

-- dashboards
```

```
CREATE TABLE dashboard (
  id TEXT PRIMARY KEY, name TEXT NOT NULL, sort INTEGER NOT NULL,
  layouts TEXT NOT NULL,           -- JSON: { phone: [...], tablet: [...],
desktop: [...] }
  updated_at TEXT NOT NULL
);
CREATE TABLE widget (
  id TEXT PRIMARY KEY, dashboard_id TEXT NOT NULL REFERENCES dashboard(id) ON
DELETE CASCADE,
  type TEXT NOT NULL, label TEXT, path TEXT, config TEXT NOT NULL DEFAULT
'{}'
);
CREATE TABLE threshold (
  id TEXT PRIMARY KEY, path TEXT NOT NULL, label TEXT NOT NULL,
  warn_low REAL, warn_high REAL, alarm_low REAL, alarm_high REAL,   -- SI
units
  enabled INTEGER NOT NULL DEFAULT 1
);

-- history (downsampled)
CREATE TABLE sample_1s  (path TEXT NOT NULL, ts INTEGER NOT NULL, value REAL,
PRIMARY KEY (path, ts)) WITHOUT ROWID;
CREATE TABLE sample_1m  (path TEXT NOT NULL, ts INTEGER NOT NULL, avg REAL,
min REAL, max REAL, PRIMARY KEY (path, ts)) WITHOUT ROWID;
CREATE TABLE sample_10m (path TEXT NOT NULL, ts INTEGER NOT NULL, avg REAL,
min REAL, max REAL, PRIMARY KEY (path, ts)) WITHOUT ROWID;
CREATE TABLE connection_gap (id TEXT PRIMARY KEY, started_at TEXT NOT NULL,
ended_at TEXT);

-- logbook
CREATE TABLE passage (
  id TEXT PRIMARY KEY, name TEXT NOT NULL, from_place TEXT, to_place TEXT,
  skipper_id TEXT REFERENCES user(id), crew TEXT NOT NULL DEFAULT '[]',
  started_at TEXT NOT NULL, ended_at TEXT
);
CREATE TABLE log_entry (
  id TEXT PRIMARY KEY, passage_id TEXT REFERENCES passage(id),
  ts TEXT NOT NULL,
  kind TEXT NOT NULL CHECK (kind IN
('hourly','event','manual','anchor','checklist','system')),
  event_type TEXT,                 -- e.g. engine_start, pressure_drop,
reef_in
  snapshot TEXT NOT NULL,          -- JSON of raw SI values, null for stale
```

```
or missing
  crew_fields TEXT NOT NULL DEFAULT '{}',  -- sea state, sky, sail plan,
watch keeper
  remarks TEXT, created_by TEXT REFERENCES user(id),
  signed_by TEXT REFERENCES user(id), signed_at TEXT
);
CREATE TABLE log_amendment (
  id TEXT PRIMARY KEY, log_entry_id TEXT NOT NULL REFERENCES log_entry(id),
  field TEXT NOT NULL, old_value TEXT, new_value TEXT, reason TEXT NOT NULL,
  user_id TEXT NOT NULL REFERENCES user(id), ts TEXT NOT NULL
);
CREATE TRIGGER log_entry_snapshot_immutable BEFORE UPDATE OF snapshot, ts,
kind, crew_fields, remarks ON log_entry
BEGIN SELECT RAISE(ABORT, 'log entries are append-only; use log_amendment');
END;
CREATE TRIGGER log_entry_no_delete BEFORE DELETE ON log_entry
BEGIN SELECT RAISE(ABORT, 'log entries cannot be deleted'); END;

-- maintenance
CREATE TABLE equipment (
  id TEXT PRIMARY KEY, system TEXT NOT NULL, name TEXT NOT NULL,
  make TEXT, model TEXT, serial TEXT, location TEXT, supplier TEXT,
  installed_on TEXT, warranty_until TEXT,
  meter_path TEXT,                 -- e.g. propulsion.port.runTime
  notes TEXT, archived INTEGER NOT NULL DEFAULT 0
);
CREATE TABLE maintenance_task (
  id TEXT PRIMARY KEY, equipment_id TEXT NOT NULL REFERENCES equipment(id),
  title TEXT NOT NULL,
  every_hours REAL, every_days INTEGER, every_nm REAL, due_on TEXT,   -- any
combination; whichever comes first
  soon_hours REAL DEFAULT 25, soon_days INTEGER DEFAULT 30,
  last_done_at TEXT, last_done_hours REAL, last_done_nm REAL,
  template_id TEXT, active INTEGER NOT NULL DEFAULT 1
);
CREATE TABLE maintenance_record (
  id TEXT PRIMARY KEY, equipment_id TEXT NOT NULL REFERENCES equipment(id),
  task_id TEXT REFERENCES maintenance_task(id),   -- null for ad-hoc repairs
  done_at TEXT NOT NULL, meter_hours REAL, user_id TEXT REFERENCES user(id),
  notes TEXT, cost_cents INTEGER
);
CREATE TABLE part (
  id TEXT PRIMARY KEY, name TEXT NOT NULL, part_number TEXT,
```

```
  qty_aboard REAL NOT NULL DEFAULT 0, min_qty REAL NOT NULL DEFAULT 0,
  unit TEXT, location TEXT, supplier TEXT
);
CREATE TABLE task_part (task_id TEXT REFERENCES maintenance_task(id), part_id
TEXT REFERENCES part(id), qty REAL NOT NULL, PRIMARY KEY (task_id, part_id));
CREATE TABLE record_part (record_id TEXT REFERENCES maintenance_record(id),
part_id TEXT REFERENCES part(id), qty REAL NOT NULL, PRIMARY KEY (record_id,
part_id));

-- shared
CREATE TABLE attachment (
  id TEXT PRIMARY KEY, owner_type TEXT NOT NULL, owner_id TEXT NOT NULL,
  file_path TEXT NOT NULL, mime TEXT NOT NULL, caption TEXT, created_at TEXT
NOT NULL
);
CREATE TABLE checklist (id TEXT PRIMARY KEY, name TEXT NOT NULL, items TEXT
NOT NULL, writes_log INTEGER NOT NULL DEFAULT 1);
CREATE TABLE checklist_run (
  id TEXT PRIMARY KEY, checklist_id TEXT NOT NULL REFERENCES checklist(id),
  started_at TEXT NOT NULL, finished_at TEXT, user_id TEXT REFERENCES
user(id),
  results TEXT NOT NULL DEFAULT '{}'
);
CREATE TABLE anchor_set (
  id TEXT PRIMARY KEY, lat REAL NOT NULL, lon REAL NOT NULL,
  rode_m REAL, radius_m REAL NOT NULL, depth_m REAL,
  set_at TEXT NOT NULL, lifted_at TEXT
);
```

Signing an entry (`signed_by` , `signed_at`) is the only update allowed on `log_entry` . Crew fields and remarks entered after creation are stored as amendments. History retention runs every 10 minutes: delete `sample_1s` older than 24 h and `sample_1m` older than 90 days, after rolling them up.

Backend API

REST under `/api/v1` , JSON only, Zod-validated in and out. One WebSocket at `/api/v1/events` carries both live instrument values and change events. The backend also serves the built web app. Everything is reached through HA ingress on port 8099, and the add-on exposes no other ports.

Auth

Ingress authenticates every request before it reaches the add-on. The backend reads the HA user from the `X-Remote-User-Id`, `X-Remote-User-Name` and `X-Remote-User-Display-Name` headers, and trusts them only on connections from the Supervisor's ingress proxy (172.30.32.2). Requests from any other address are rejected.

Method	Path	Notes
GET	`/auth/me`	HA user, mapped role, unit prefs
GET, PUT	`/users`	Skipper maps HA users to roles. Unmapped HA users are guests
POST	`/auth/pin`	Optional: a shared helm device signed in as a guest HA user can elevate to crew with a PIN for 12 h
GET	`/health`	Used by the HA watchdog: process up and database writable. Does not fail when Signal K is down

Roles: guest reads dashboards only. Crew can also create log entries, sign, amend, and complete maintenance tasks. Skipper can do everything, including settings, role mapping, thresholds, deleting equipment and editing dashboards. Enforce in a Fastify `preHandler`, with a test per route.

Resources

Area	Endpoints
Dashboards	`GET/POST /dashboards`, `GET/PUT/DELETE /dashboards/:id`, `PUT /dashboards/:id/layouts`, `POST /dashboards/import`, `GET /dashboards/:id/export`
Paths	`GET /paths` (catalog merged with live Signal K paths, current value, age)
History	`GET /history?paths=a,b&from=&to=&maxPoints=` (picks the resolution tier automatically)

Area	Endpoints
Thresholds	`GET/POST /thresholds`, `PUT/DELETE /thresholds/:id`
Alarms	`GET /alarms/active`, `POST /alarms/:id/ack`, `POST /alarms/:id/snooze {minutes}`
Passages	`GET/POST /passages`, `GET/PUT /passages/:id`, `POST /passages/:id/end`, `GET /passages/:id/summary`
Log entries	`GET /log?passageId=&kind=&unsigned=&before=&limit=`, `POST /log` (manual), `GET /log/:id`, `POST /log/:id/sign`, `POST /log/:id/amend {field, newValue, reason}`
Log export	`GET /passages/:id/export.pdf`, `.csv`, `.gpx`, `GET /export/full.json`
Equipment	`GET/POST /equipment`, `GET/PUT/DELETE /equipment/:id`
Tasks	`GET /tasks/due` (sorted by urgency, with computed status), `POST /equipment/:id/tasks`, `PUT/DELETE /tasks/:id`, `POST /tasks/:id/complete`
Records	`GET /equipment/:id/records`, `POST /equipment/:id/records` (ad-hoc repair)
Parts	`GET/POST /parts`, `PUT/DELETE /parts/:id`, `GET /parts/shopping-list`
Pre-departure	`GET /maintenance/pre-departure?hours=&days=`
Anchor	`GET /anchor`, `POST /anchor/drop {radiusM, rodeM?}`, `PUT /anchor {radiusM}`, `POST /anchor/lift`
Checklists	`GET/POST /checklists`, `PUT/DELETE /checklists/:id`, `POST /checklists/:id/runs`, `PUT /checklist-runs/:id`

Area	Endpoints
Attachments	`POST /attachments` (multipart, resized to 2048 px max edge), `GET /attachments/:id`
System	`GET /system/status` (Signal K connection, `time source and GPS drift, CPU temperature, disk, last snapshot`)

WebSocket events

Client to server: `{type: "live.subscribe", paths: [...]}` replaces that client's live subscription. Server to client: `{type: "live", values: {<path>: [value, ts]}}`, batching every changed path the client asked for, at most 4 times a second. Other server-to-client events use `{type, payload}`:

Type	When
`alarm.raised`, `alarm.cleared`, `alarm.acked`	Any threshold, anchor or system alarm changes state
`log.created`, `log.signed`, `log.amended`	Any log change
`passage.suggestStart`, `passage.suggestEnd`	Auto-detection fires (see Logbook)
`passage.started`, `passage.ended`	
`dashboard.updated`	Layout or widget saved by another client
`task.statusChanged`	A task crosses into due soon, due or overdue
`system.status`	Signal K connected or lost, clock sync changed

Frontend

One React app and one route tree, served under HA ingress. Ingress mounts the app at a path HA chooses, so every asset, route, API call and WebSocket URL is built from that base: relative Vite `base`, a router basename read at startup, and no absolute `/` paths anywhere. On phones it runs inside the HA companion app. At the nav station it runs in

any browser signed in to HA. The shell switches between bottom tabs and the sidebar by breakpoint, and every screen works at every width. Screens marked with a mockup should match it closely. The rest follow the same tokens and components.

Routes

Route	Screen	Mockup
`/login`	Guest-mode PIN pad for shared helm devices, 56 px keys. Everyone else arrives signed in through ingress	None
`/`	Redirects to the last-viewed dashboard	
`/d/:dashboardId`	Dashboard, view mode. Swipe between dashboards on phone, tabs on desktop	Helm dashboard (day), Helm dashboard (night)
`/d/:dashboardId/edit`	Dashboard edit mode with widget settings and path browser	Desktop dashboard, edit mode
`/log`	Logbook timeline for the active passage, or the most recent	Logbook timeline
`/log/new`	Quick entry	Quick log entry
`/log/:entryId`	Entry detail: snapshot, crew fields, sign, amend, amendment history	None
`/passages`, `/passages/:id`	Passage list and summary: track, stats, timeline, export	None
`/maintenance`	Due list	Maintenance due list

Route	Screen	Mockup
`/maintenance/equipment`, `/maintenance/equipment/:id`	Equipment list and detail, including complete-task form	Desktop equipment detail
`/maintenance/parts`	Inventory and shopping list	None
`/maintenance/pre-departure`	Everything due within a planned passage	None
`/anchor`	Anchor watch: set, watching, alarm states	Anchor watch, dragging
`/checklists`, `/checklists/:id/run`	Checklist list and run	None
`/more`	Phone-only menu: Anchor, Checklists, Passages, Parts, Settings	None
`/settings`	Units, night mode schedule, log interval and triggers, thresholds, role mapping, Signal K instance ids, system status	None
`/setup`	First run: Signal K device approval, boat details, mapping HA users to roles, units, starter dashboards	None

Shell behavior

- **Night mode.** Toggle from the moon button on every screen. Optional auto-switch at local sunset and sunrise, computed from `navigation.position` with a small solar calculator (no network). The theme is stored per device in `localStorage`, not per user, because the night helm phone and the saloon laptop want different settings.

- **Alarm banner.** Driven by `alarm.raised` events and `/alarms/active` on load. Full-width, top of the screen, pushes content down. Acknowledge and Snooze buttons at 56 px. Plays a tone through Web Audio while unacknowledged and the page is visible. For alarm-class events the backend also sends an HA companion app notification, critical on iOS. Neither is the real alarm. The onboard sounder is.
- **Status strip.** Data Hub connection, Signal K connection, GPS fix, time (with its source: GPS or host), passage state. Tapping it opens system status in settings.
- **No PWA.** Ingress and service workers don't mix, and the companion app already gives a home-screen app. The app lives on the boat LAN, so no internet is the normal case, not an error. If the Data Hub WebSocket drops, show a non-blocking "Reconnecting" strip, keep the last values on screen, and let them go stale as their timeouts pass.
- **Screen wake.** Request a Screen Wake Lock on dashboard and anchor routes when the user turns on "Keep screen on" in settings.

Dashboard rendering

- A live store (`useLiveValue(path)`) holds the latest value and timestamp per path, fed by `live` messages from the Data Hub WebSocket. When the set of paths needed by mounted widgets changes, the store sends a new `live.subscribe`.
- The server batches updates to at most 4 per second per client, so widgets never re-render faster than that.
- Widget types and their configs are defined in a registry (`dashboards/widgets/index.ts`), so adding a widget is one file plus a registry entry.

Accessibility

Real buttons, links and labelled inputs everywhere. Icon-only buttons need `aria-label`. Text meets 4.5:1 contrast in day mode. Night mode is deliberately low contrast: it's exempt, and that's documented in `DECISIONS.md`. The alarm banner uses `role="alert"`.

Module specs

These are the behavior rules the tests check. Anything with a default value is configurable in settings by the skipper.

Dashboards

Widget library

Type	Config	Notes
`numeric`	path, decimals, show min/max since reset	Default widget for any number
`gauge`	path, min, max, warn and alarm zones	Semicircle arc, as in the edit-mode mockup
`wind`	apparent and true angle and speed paths	Boat-relative rose, close-hauled sectors 20° to 60° each side, catamaran outline in the center
`heading`	path, true or magnetic	Value plus small tape
`depth`	path, offset, shallow threshold	"No bottom" when out of range
`trend`	1 to 4 paths, window 10 min to 7 days	uPlot, from `/history`, appends live values
`twin`	a list of port/starboard path pairs	Port solid, starboard dashed. Labels PORT and STBD
`battery`	voltage, current, SoC, solar paths	
`tank`	level path, capacity	Vertical fill bar, volume shown in user units
`position`	none	Lat/lon, SOG, COG
`text`	path	Non-numeric paths
`clock`	UTC, local, countdown	
`link`	URL, label	Opens Freeboard-SK, the Signal K admin, and so on

Layouts. Each dashboard stores a separate `react-grid-layout` layout per breakpoint: phone 2 columns, tablet 6, desktop 12. A widget missing from a breakpoint's layout is auto-placed at the bottom. Edit mode shows drag handles and resize corners. The breakpoint being edited is chosen with a segmented control, as in the mockup. View mode is locked: no drag, no resize.

Starter dashboards. Setup creates Helm, Engines, Energy and Anchor, using only the paths Signal K is actually publishing.

Derived values. Computed in the backend at 1 Hz, published to Signal K and recorded like any other path:

Value	Formula
Total fuel rate	sum of `propulsion.*.fuel.rate`
Fuel range	fuel tank volume ÷ total fuel rate × SOG
Net battery current	house battery current (already net if the shunt is on the negative)
Time to empty	(SoC − floor SoC) × capacity ÷ discharge current, only while discharging. Floor SoC defaults to 50%
Distance run	sum of great-circle distances between successive positions, ignoring jumps over 1 nm in 1 s
24-hour run	distance run over the trailing 24 h
Set and drift	vector difference of COG/SOG and heading/STW, when all four are fresh
Pressure tendency	change in `environment.outside.pressure` over the last 3 h

Thresholds and alarms. Evaluated in the backend on every update. Use hysteresis: an alarm clears only once the value is back past the threshold by 2% of the range for 10 s. Raising writes a `notifications.app.*` delta, emits `alarm.raised`, and writes an event log entry. Acknowledge silences sound but keeps the banner until the condition clears. Snooze silences for N minutes, then re-raises if the condition still holds.

Logbook

Hourly entries. At minute 0 of every UTC hour while a passage is active, the backend builds a snapshot from current values and writes a `hourly` entry. With no passage active, the interval is 4 hours if `navigation.anchor.position` or an anchor set is active, and off otherwise. If the backend was down at the top of the hour, it doesn't invent the missing entry. It writes a `system` entry on restart ("No entries 14:00 to 16:00: data hub offline").

Time source. HAOS gives an add-on no control over the host clock, and offshore there is no network time. The logbook therefore runs on GPS time. Every entry's `ts` comes from Signal K's `navigation.datetime`, and the hourly scheduler fires on GPS-time hour

boundaries. If GPS time is stale, fall back to the host clock and record `clock_source: "host"` in the snapshot. Once a minute the backend compares GPS and host time, and raises a warning alarm if they differ by more than 5 s.

Snapshot fields. Position, COG, SOG, heading, STW, distance this hour, distance since departure, TWS, TWD, AWA, pressure, pressure tendency, air and water temperature, depth below keel, each engine's state, rpm and hours, fuel level, house voltage and SoC, solar current, tank levels, autopilot state. Stale values are `null`. Crew fields (sea state, sky, visibility, sail plan, watch keeper) carry forward from the previous entry and are shown pre-filled, marked as carried forward.

Event triggers

Trigger	Default	Debounce
Engine start or stop	Any `propulsion.*.state` change, or rpm crossing 300	30 s
Course change	More than 30° from the 5-minute average COG	Held 5 min
Wind change	TWS change over 10 kn within 60 min	One per hour
Pressure drop	3 h tendency below −2 hPa	One per 3 h
Alarm	Any raise or acknowledge	None
Anchor	Drop and lift	None
Waypoint arrival	Signal K course API arrival, if available	None

Passage auto-detection. Suggest start when SOG stays above 2 kn for 10 min and no passage is active. Suggest end after anchor down or 30 min below 0.5 kn. Suggestions arrive as a prompt; they never start or end a passage on their own.

Signing and amending. Unsigned entries show the amber UNSIGNED label. Signing records who and when. Any later change is an amendment with a required reason, displayed under the entry with the original value struck through.

Export. PDF in traditional logbook layout, one page per UTC day, landscape, columns for time, position, course, speed, wind, barometer, sea state, sail plan, remarks and signature. CSV with every snapshot field in user units. GPX track of 1-minute positions. JSON full backup.

Maintenance

Due calculation, for each active task, computed on read and every 10 minutes for status-change events:

```
remaining_hours = (last_done_hours + every_hours) − current meter hours
remaining_days  = (last_done_at + every_days) − today
remaining_nm    = (last_done_nm + every_nm) − distance logged since
status = overdue   if any remaining < 0
       = due       if any remaining_hours ⩽ 5 or remaining_days ⩽ 14
       = due_soon  if any remaining_hours ⩽ soon_hours or remaining_days ⩽
soon_days
       = ok        otherwise
```

The mockup labels "due" as "Due now". A task with no history uses the equipment's install date and zero hours. If the meter path is stale, use the last recorded value and show "meter stale" on the task.

Completing a task pre-fills current meter hours, decrements stock for each linked part, writes a record, resets the task's last-done fields, and adds any part that drops below its minimum to the shopping list. Show the stock warning inline before the user confirms, as in the equipment mockup.

Templates. Ship JSON templates for diesel engine, saildrive, outboard, watermaker, generator, standing rigging, life raft, EPIRB, flares, fire extinguishers and first aid. Every interval in a template is a placeholder the skipper confirms from the manufacturer's manual during setup. Don't ship invented intervals as if they were authoritative.

Anchor watch

- Drop: record current position (or a position the user drags on the swing-circle view), radius (default rode length plus boat length, editable), rode and depth.
- Distance is the haversine distance from the anchor to the GPS antenna position. A setting adds an antenna-to-bow offset.
- Alarm when 3 consecutive fixes over 10 s are outside the radius. A single bad fix never triggers.
- Also alarm when position goes stale for 30 s while the anchor watch is on. Losing GPS at anchor is itself a hazard.
- Sound: raise the Signal K notification and drive the onboard sounder. The sounder is a pluggable output: GPIO pin via `libgpiod`, or a configurable shell command. Unit-test the output interface. Integration-test it on the Pi.

- The phone screen shows the swing circle, a breadcrumb track of the last 60 min, distance, bearing, wind and depth, as in the mockup.

Checklists

Each checklist is an ordered list of items with optional notes. A run records each item as done, skipped or failed, with the time. Finishing a run writes a `checklist` log entry if `writes_log` is on. Ship starter lists (pre-departure, arrival, night watch handover, heavy weather, abandon ship, engine start) as editable templates with items for the skipper to review.

Ops

HAOS owns the operating system, the container runtime, updates and backups. The add-on only has to behave well inside it. There's no install script: add the Data Hub repository in HA, install the add-on, set its options, start it.

Hardware and heat

- **Pi 5.** 8 GB, NVMe boot, in a large passive heatsink case. No fan: a fan pulls salt air across the board. Consider conformal coating.
- **Or a fanless x86 mini PC** with 12 V input if the Pi runs out of headroom. More margin, at roughly 6 to 15 W against 3 to 7 W for the Pi.
- Mount it in a ventilated, dry spot such as the nav station. Never in the engine room or a sealed locker.
- Watch the temperature. A Pi 5 starts throttling around 80°C. Use HA's system monitor sensor for CPU temperature, and have the add-on read it and alarm at 70°C (warning) and 78°C (alarm), well before throttling.

Keeping the CPU down

- Sample rates and batched writes as described in the Signal K section.
- Images are prebuilt in CI. Nothing compiles on the device.
- PDF export runs on demand only. No background rendering.
- Steady-state target: under 10% of one Pi 5 core with every instrument connected. Measured in M6.

- **Keep instrument data out of HA's recorder.** The add-on publishes only these summary entities to HA, on change and at most once a minute (alarms immediately): `sensor.data_hub_active_alarm`, `sensor.data_hub_passage_state`, `sensor.data_hub_next_maintenance`, `binary_sensor.data_hub_anchor_watch`, `sensor.data_hub_house_battery`. The README tells the skipper to trim HA itself, for example:

```
recorder:
  commit_interval: 30
  purge_keep_days: 7
  exclude:
    entity_globs:
      - sensor.signalk_*
```

Watchdog, power and sounder

- HA's add-on watchdog calls `/api/v1/health` and restarts the add-on if it fails. Health means the process is up and the database is writable. Losing Signal K must not trigger restarts.
- UPS HAT: the add-on watches the GPIO pin in the `ups_gpio` option. On low input voltage it writes a `system` log entry, checkpoints the WAL, then asks the Supervisor to shut the host down (`POST http://supervisor/host/shutdown` with `SUPERVISOR_TOKEN`).
- Sounder: drives the GPIO pin in the `sounder_gpio` option. Built behind an output interface so a different sounder can be swapped in.

Time

HAOS syncs its clock over the network when it can, and the add-on can't change that. Fit the Pi 5 RTC battery so the clock survives a cold boot. Everything that needs trustworthy time (the logbook) uses GPS time, as described in Module specs.

Backups

- HA's automatic backups include the add-on, using the hot backup and snapshot approach described in Stack and packaging. The skipper points HA's backups at a USB stick or another location HA supports.
- Restore is an HA restore. CI tests that a snapshot taken while logging restores to an identical, consistent database.

Updates

The add-on repository lives on GitHub, with versioned images on GHCR. HA shows the update and can take a backup first. Updates need internet, so plan them at the dock. Rollback is restoring the pre-update backup. Migrations must be forward-only and tested against a copy of a real database before release.

Security

- Ingress only. No ports exposed on the host. User headers are trusted only from the ingress proxy address.
- The Signal K device token lives in `/data` with `0600` permissions.
- Remote access, if wanted, uses HA's own options (Nabu Casa or a Tailscale add-on). Documented, not required.
- Guest PINs hashed with argon2id. Lock for 5 minutes after 5 bad PINs.
- Boat documents and crew personal data, when that module ships, are encrypted at rest.

Milestones and acceptance criteria

Six milestones, built in order. Each ends with its criteria passing in CI against the replay fixtures. Items marked "on boat" are checked by the skipper on the real hardware.

M1: Foundation

Monorepo, shared package, add-on packaging and CI multi-arch images, Signal K client with reconnect, rate classes and staleness, buffered history recorder and downsampling, SQLite migrations, ingress auth and role mapping, shell (tabs, sidebar, status strip, night mode, alarm banner slot), path browser.

- [ ] Replay of a 2-hour fixture produces the expected row counts per tier: fast-class paths in `sample_1s`, everything in `sample_1m` and `sample_10m`. Retention prunes correctly.
- [ ] Writes land in batches every 10 s, except log entries and alarms, which are immediate.
- [ ] Killing the fake Signal K server for 60 s records one `connection_gap`, and every path goes stale in the UI within its effective timeout.
- [ ] Every function in `units.ts` passes round-trip tests.
- [ ] The outbound-path allowlist test passes.

- [ ] Requests carrying user headers from any address other than the ingress proxy are rejected. Guest gets 403 on every write route.
- [ ] The app loads, navigates and streams live values under a random ingress-style path prefix. Shell matches the mockups at 390 px and 1440 px (Playwright screenshot review).
- [ ] Installs from the add-on repository on a fresh HAOS VM with no build step on the device.
- [ ] On boat: 7 days of recording with no unexplained gaps, a clean recovery from pulling the power cable, and CPU temperature logged the whole time.

M2: Dashboards

Widget library, layouts per breakpoint, edit mode, path browser binding, derived values, thresholds and alarms, starter dashboards.

- [ ] Phone helm and desktop edit-mode screens match the mockups in day and night.
- [ ] A layout edited on desktop doesn't change the phone layout.
- [ ] Two browsers: a layout saved in one appears in the other within 2 s.
- [ ] Threshold fixture: alarm raises once, hysteresis prevents flapping, ack silences, snooze re-raises.
- [ ] Live value appears within 500 ms of the delta. Widget re-render capped at 4 Hz.
- [ ] Derived values match hand-calculated expected values from the fixture.

M3: Logbook

Passages, hourly scheduler, snapshot builder, event triggers, quick entry, sign and amend, passage summary, PDF/CSV/GPX export.

- [ ] A 6-hour accelerated replay with a passage active produces exactly 6 hourly entries at :00 UTC, plus the expected event entries and no duplicates.
- [ ] A fixture with a dead sensor produces `null` for that field, never a repeated value.
- [ ] Backend stopped across a top-of-hour: on restart, one `system` entry, no back-filled hourly entry.
- [ ] Direct SQL `UPDATE` of a snapshot or `DELETE` of an entry fails.
- [ ] Amendment shows original and new values with reason.
- [ ] PDF renders one page per UTC day with every column. CSV and GPX open in standard tools.

- [ ] Logbook timeline and quick entry match the mockups.
- [ ] On boat: an overnight passage with a complete hourly log, signed and exported, and no screen touched to create the hourly entries.

M4: Maintenance

Equipment, tasks, due calculation, completion, parts and inventory, shopping list, templates, pre-departure view.

- [ ] Due-calculation unit tests cover every branch, including stale meter and tasks with no history.
- [ ] Completing a task with linked parts decrements stock and adds below-minimum parts to the shopping list.
- [ ] `task.statusChanged` fires when replayed engine hours cross a threshold.
- [ ] Due list and equipment detail match the mockups.
- [ ] On boat: an oil change on both engines tracked end to end.

M5: Anchor watch and checklists

- [ ] Drag fixture: alarm after 3 consecutive out-of-radius fixes, not on a single spike.
- [ ] GPS-loss fixture: alarm after 30 s stale with watch on.
- [ ] Sounder interface tested with a fake output. On boat: buzzer sounds with every phone locked.
- [ ] Anchor watch screen matches the mockup, including the night alarm state.
- [ ] Checklist run writes a log entry when enabled.

M6: Hardening

- [ ] An HA backup taken while logging restores into a fresh HAOS VM with an intact, consistent database.
- [ ] Update and rollback through HA tested.
- [ ] GPS-time fixture: with the host clock deliberately wrong, log timestamps follow `navigation.datetime` and the drift warning raises past 5 s.
- [ ] UPS low-voltage signal triggers a clean host shutdown through the Supervisor, with a log entry.
- [ ] 6 concurrent clients for 1 hour with no missed hourly entry and backend memory stable.

- [ ] Steady state under 10% of one Pi 5 core with full instrument load, and no thermal throttling in a 24-hour soak inside the planned enclosure.

Stop and ask

Stop and ask the skipper before doing any of these. Don't guess and continue.

- Writing to any Signal K path outside the allowlist, or anything that could reach the NMEA bus.
- Changing the append-only rules on `log_entry`, or any migration that rewrites existing log data.
- Adding a runtime dependency on the internet, a CDN, or a cloud service.
- Publishing more HA entities than the summary set, or anything that puts instrument data into HA's recorder.
- Exposing a port outside ingress, or raising a path's sample rate class.
- Adding a color, font or component style not in the design system.
- Choosing default maintenance intervals or alarm thresholds that affect safety (anchor radius, depth, engine temperature). Ship them as placeholders the skipper confirms.
- Moving alarm evaluation into the browser for any reason.
- Replacing SQLite, Fastify, React or uPlot.

Also check with the skipper on these open items, ideally before M2:

- [ ] The real Signal K instance ids for engines, batteries and tanks once the instruments are installed.
- [ ] Battery monitor and tank senders, which decide what the Energy dashboard can show on day one.
- [ ] Sounder hardware: GPIO buzzer, speaker, or MFD only.
- [ ] Whether the insurer or flag state requires a specific logbook format.
- [ ] Cloud backup target, if any.